

Stage 3: Production-Grade RBAC & Permission Matrix Specification

Alight International Multi-Vendor Marketplace *Document Version: 3.0.0 (Production-Ready)* *Security Architecture:* Stateless JWT Bearer Authentication + Spring Security 6 Method Security + Multi-Tenant Resource Isolation *Database Authority:* PostgreSQL `roles` , `permissions` , `role_permissions` , `user_roles` (Flyway `V2` , `V31`)

1. 4-Tier Role Architecture

Role Identifier	Display Name	Target Persona	Scope of Authority
<code>ROLE_CUSTOMER</code>	Registered Customer	Shoppers, Architects, B2B Contractors	Personal cart, orders, addresses, reviews, Q&A, and return requests.
<code>ROLE_VENDOR</code>	Verified Vendor / Seller	Hardware manufacturers, boutique brass studios	Tenant-isolated store profile, catalog, stock, sub-orders, wallet & returns.
<code>ROLE_ADMIN</code>	Platform Operations Admin	Marketplace moderators, catalog managers	System-wide catalog moderation, vendor KYC approval, dispute resolution.
<code>ROLE_SUPER_ADMIN</code>	Super Administrator	Executive leadership, Lead DevOps	Full unrestricted access, user role mutation, financial ledger & audit trail.

2. Comprehensive 4-Tier Role-Permission Matrix (45 Granular Permissions)

Domain	Granular Permission Key	Description	Customer	Vendor	Admin	Super Admin
Profile	user:profile_manage	Read and update personal profile	✓	✓	✓	✓
Profile	user:address_manage	Manage personal shipping addresses	✓	✓	✓	✓
Catalog	catalog:read	Browse active catalog & categories	✓	✓	✓	✓
Catalog	catalog:create	Create new draft products & variants	✗	✓	✓	✓
Catalog	catalog:update	Edit owned product details & prices	✗	✓ (Tenant)	✓	✓
Catalog	catalog:delete	Archive/delete owned products	✗	✓ (Tenant)	✓	✓
Catalog	catalog:approve	Approve/reject vendor submissions	✗	✗	✓	✓
Catalog	catalog:publish	Publish categories & brand taxonomy	✗	✗	✓	✓
Inventory	inventory:read	View warehouse stock & reserves	✗	✓ (Tenant)	✓	✓
Inventory	inventory:update	Update owned stock on-hand	✗	✓ (Tenant)	✓	✓
Inventory	inventory:adjust	Stock reconciliation & threshold adjustments	✗	✓ (Tenant)	✓	✓
Inventory	inventory:reserve	Temporary checkout stock reservation	✗	✗	✓	✓
Orders	order:read_own	View personal placed orders	✓ (Own)	✗	✓	✓
Orders	order:read_vendor	View assigned vendor sub-orders	✗	✓ (Tenant)	✓	✓

Domain	Granular Permission Key	Description	Customer	Vendor	Admin	Super Admin
Orders	order:read_all	View all platform master & sub-orders	✗	✗	✓	✓
Orders	order:update_status	Update fulfillment status (SHIPPED)	✗	✓ (Tenant)	✓	✓
Orders	order:cancel	Cancel customer order / out-of-stock item	✓ (Own)	✗	✓	✓
Vendor	vendor:profile_read	View vendor business details	✗	✓ (Tenant)	✓	✓
Vendor	vendor:profile_update	Update store logo, description, pickup	✗	✓ (Tenant)	✓	✓
Vendor	vendor:kyc_submit	Submit GSTIN, PAN, bank verification	✗	✓ (Tenant)	✓	✓
Vendor	vendor:verify_approve	Review & approve vendor KYC	✗	✗	✓	✓
Vendor	vendor:commission_update	Configure custom commission rate per vendor	✗	✗	✓	✓
Settlement	settlement:wallet_read	View wallet balance & escrow holds	✗	✓ (Tenant)	✓	✓
Settlement	settlement:payout_request	Request withdrawal from wallet	✗	✓ (Tenant)	✓	✓
Settlement	settlement:payout_approve	Review & approve payout requests	✗	✗	✓	✓
Settlement	settlement:payout_process	Execute bank disbursement batches	✗	✗	✗	✓
Support	support:ticket_create	Open new support / dispute ticket	✓	✓	✓	✓
Support	support:ticket_reply	Post message in dispute discussion	✓ (Own)	✓ (Own)	✓	✓

Domain	Granular Permission Key	Description	Customer	Vendor	Admin	Super Admin
Support	support:ticket_assign	Route tickets to specific agents	✗	✗	✓	✓
Support	support:ticket_resolve	Close or escalate dispute tickets	✗	✗	✓	✓
Reviews	review:create	Submit verified purchase review	✓	✗	✓	✓
Reviews	review:vote	Upvote review helpfulness	✓	✓	✓	✓
Reviews	review:moderate	Moderate/delete inappropriate reviews	✗	✗	✓	✓
QA	qa:ask	Post pre-purchase product question	✓	✓	✓	✓
QA	qa:answer	Answer question for owned product	✗	✓ (Tenant)	✓	✓
QA	qa:moderate	Moderate inappropriate Q&A threads	✗	✗	✓	✓
Coupons	coupon:create_vendor	Create vendor-scoped discount voucher	✗	✓ (Tenant)	✓	✓
Coupons	coupon:create_global	Create marketplace-wide coupon campaign	✗	✗	✓	✓
Coupons	coupon:manage_all	Activate, edit, or deactivate any promo	✗	✗	✓	✓
Logistics	logistics:manifest_create	Generate carrier pickup manifests	✗	✓ (Tenant)	✓	✓
Logistics	logistics:label_generate	Generate AWB shipping labels	✗	✓ (Tenant)	✓	✓
Logistics	logistics:carrier_manage	Configure carriers & pincode zone rates	✗	✗	✓	✓
Returns	returns:request_create	Submit return/RMA with photo proof	✓ (Own)	✗	✓	✓

Domain	Granular Permission Key	Description	Customer	Vendor	Admin	Super Admin
Returns	<code>returns:vendor_action</code>	Accept, replace, or dispute return	✗	✓ (Tenant)	✓	✓
Returns	<code>returns:admin_override</code>	Force immediate customer refund	✗	✗	✓	✓
Returns	<code>returns:inspect</code>	Warehouse grading & inspection report	✗	✗	✓	✓
Analytics	<code>analytics:vendor_dashboard</code>	View seller GMV, conversions & returns	✗	✓ (Tenant)	✓	✓
Analytics	<code>analytics:platform_executive</code>	View marketplace GMV & tax ledgers	✗	✗	✓	✓
System	<code>system:user_role_manage</code>	Assign & revoke user roles/permissions	✗	✗	✗	✓
System	<code>system:audit_read</code>	Query immutable audit logs	✗	✗	✓	✓
System	<code>system:config_manage</code>	Configure currencies, taxes, gateways	✗	✗	✗	✓

3. Multi-Tenant Resource Isolation Architecture

Tenant Security Principles

1. **Zero Cross-Vendor Leakage:** A Vendor can strictly access and mutate catalog items, warehouse stock records, fulfillment sub-orders, and wallet transactions where `vendor_id = current_vendor_id`.
2. **Customer Isolation:** Customers cannot view or mutate orders, dispute tickets, or saved addresses belonging to other user accounts.
3. **Admin Escalation:** Platform Administrators and Super Administrators possess supervisory read/write permissions across all tenant boundaries.

Spring Security Evaluator (`ResourceSecurityService`)

JAVA

```
// Example Controller Enforcement
@PutMapping("/products/{id}")
@PreAuthorize("@securityService.isVendorOwnerOfProduct(#id) or hasRole('ADMIN')")
public ResponseEntity<ApiResponse<ProductDto>> updateProduct(@PathVariable UUID id, ...) { ... }

@GetMapping("/vendor/orders/{subOrderId}")
@PreAuthorize("@securityService.isVendorOwnerOfOrder(#subOrderId) or hasRole('ADMIN')")
public ResponseEntity<ApiResponse<VendorOrderDetailDto>> getVendorOrder(@PathVariable UUID subOrderId) { ... }
```

4. Custom Security Annotation Cheatsheet

Annotation	Target Role Authority	Usage Scenario
<code>@RequireSuperAdmin</code>	<code>ROLE_SUPER_ADMIN</code>	User role assignment, financial payout processing, audit deletion guards
<code>@RequireAdmin</code>	<code>ROLE_ADMIN</code> , <code>ROLE_SUPER_ADMIN</code>	Catalog moderation, KYC verification, refund overrides
<code>@RequireVendor</code>	<code>ROLE_VENDOR</code>	Vendor store management, dispatch manifests, product creation
<code>@RequireCustomer</code>	<code>ROLE_CUSTOMER</code>	Customer checkout, review submission, address management
<code>@RequireVendorOrAdmin</code>	<code>ROLE_VENDOR</code> , <code>ROLE_ADMIN</code> , <code>ROLE_SUPER_ADMIN</code>	Product catalog viewing, logistics tracking

5. JWT Stateless Token Payload Structure

JSON

```
{
  "sub": "seller@alight.com",
  "userId": "a0000000-0000-0000-0000-000000000002",
  "firstName": "Vikram",
  "lastName": "Sharma",
  "roles": ["ROLE_VENDOR", "ROLE_CUSTOMER"],
  "permissions": [
    "user:profile_manage",
    "catalog:read",
    "catalog:create",
    "catalog:update",
    "inventory:read",
    "order:read_vendor",
    "vendor:profile_read",
    "settlement:wallet_read"
  ],
  "iat": 1726135200,
  "exp": 1726221600
}
```

6. RBAC REST API Specification

1. GET /api/v1/admin/rbac/roles

- **Security:** @RequireAdmin
- **Response:** List of all 4 roles with full list of bound permission metadata.

2. GET /api/v1/admin/rbac/permissions

- **Security:** @RequireAdmin
- **Response:** Complete catalog of all 45 granular permissions categorized by domain (catalog , order , vendor , settlement , etc.).

3. GET /api/v1/admin/rbac/users/{userId}/roles

- **Security:** @RequireAdmin
- **Response:** Current roles, effective permissions, and vendor tenancy binding for target user.

4. PUT /api/v1/admin/rbac/users/{userId}/roles

- **Security:** @RequireSuperAdmin
- **Request Body:** { "roles": ["ROLE_VENDOR", "ROLE_CUSTOMER"] }
- **Response:** Updated user role profile and newly resolved permissions.

5. GET /api/v1/users/me/permissions

- **Security:** authenticated()
- **Response:** Self-introspection of authenticated caller's roles, permissions list, and vendor ID.